

Executive Summary

Problem: Merchants lose substantial revenue to **involuntary churn** from failed payments, abandoned checkouts, and overdue invoices. Current tools (dunning emails, static retries) recover only half of this leakage [29†L131-L139] [12†L72-L80]. We propose a unified *AI-powered Revenue-at-Risk Agent* that treats each failure event (failed charge, abandoned cart, overdue receivable) as a “case” and chooses among actions (retry, new link, reminder, escalate, stop) by maximizing **Expected Net Recovery**. Unlike existing products, our agent makes **economic decisions with stopping rules** across all case types.

Key innovations:

- **Economic Model:** Compute $\text{Expected Net Recovery} = p_{\text{success}} \times \text{Value} - \text{Cost} - \text{Friction} - \text{Risk}$. The agent uses payment status, customer value (e.g. lifetime value or invoice amount), retry history, and cost estimates to score options and decide whether it's worth continuing or stopping [29†L131-L139] [38†L72-L80].
- **Unified Cases:** Collapses “failed payment retries”, “abandoned cart campaigns” and “past-due invoices” into one workflow. Action choice is driven by a single formula, not separate tools for each scenario.
- **Data-Driven Timing:** Uses ML or heuristics to choose *when* to retry or contact, similar to Stripe’s Smart Retries and Recurly’s Intelligent Retries [38†L72-L80] [34†L26-L34], but adds a global stopping rule and cross-case optimization.
- **Auditability:** Every decision (retry, notify, stop) is logged with its inputs and score, satisfying the Razorpay requirement for explainability and gating high-risk actions.

Business Value: By front-loading intelligence and applying network-aware retry logic, empirical studies show recovery can jump ~20 percentage points [29†L131-L139] (from ~53% to ~71%) and unlock millions in revenue. We measure success directly in ₹ recovered per batch, making impact clear.

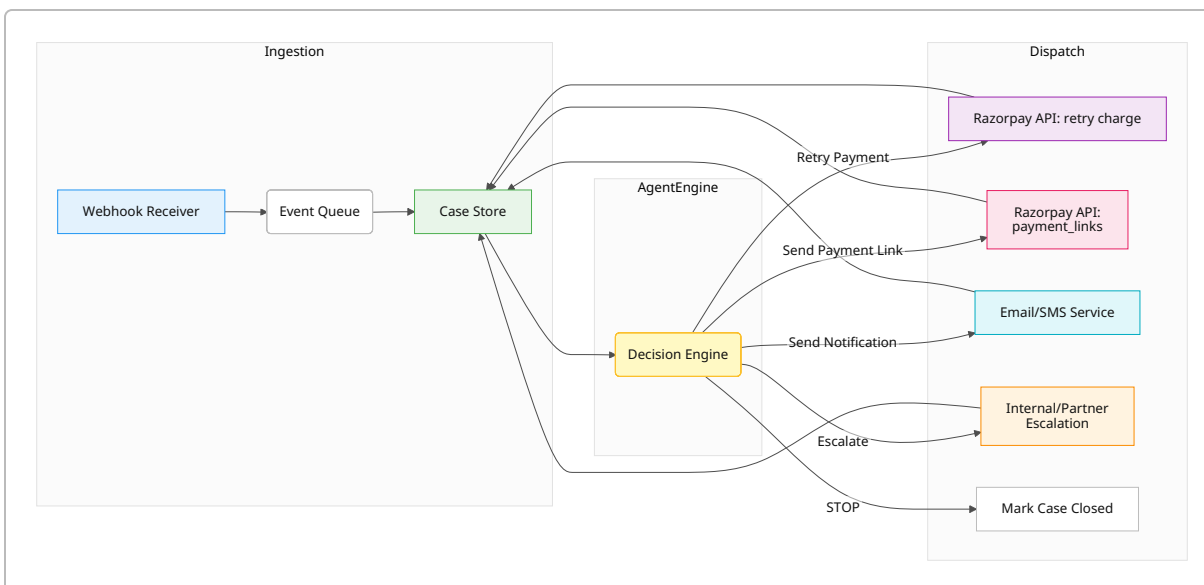
Next Steps: We build a minimal prototype that ingests webhook events (failed payment, invoice.created, etc.), stores “cases”, computes scores, invokes Razorpay APIs (retry, Payment Links), and logs outcomes. We focus on core engineering: a decision engine, robust webhook handling (idempotent, audited), and batch evaluation.

{{ Must-Build: Core decision engine with expected-value scoring (including stopping rule), webhook-driven state machine, Razorpay API integration for retry and payment link. **Do Not Build:** Complex ML training pipelines, large LLM components beyond simple scoring, fancy UI or unrelated agent features. }}

1. Technical Architecture

The system has four main layers: **Event Ingestion** → **Case Store** → **Decision Engine** → **Action Dispatcher**. Webhooks from Razorpay (failed payments, expired invoices, abandoned-cart triggers) feed into a queue

and normalize into a “RevenueRiskCase” record in our database. The Decision Engine periodically (or on-event) scans open cases, computes the **Expected Net Recovery** for each allowed action, and selects the best action or stops. The Action Dispatcher executes the chosen action via Razorpay APIs (retry charge, create/send Payment Link, send email/SMS via third-party), then updates the case. All events and decisions are logged for audit.



State Machine (Case Lifecycle): Each case transitions through states like *Open* → *Notified/Retrial Scheduled* → *Retrying...* → *Resolved or Stopped*. For example, a failed payment case goes: “New” → “Scheduled for retry” → if retry succeeds “Recovered/Closed”, if fails “Final Retry” → if still fails “Closed/Abandoned”. An explicit STOP action forces closure (customer receives no more attempts). (A mermaid state diagram would show these transitions.)

Components

- **Webhook Receiver:** Authenticates Razorpay webhooks (using signature verification [2†L145-L153] [3†L212-L220]) and enqueues events reliably (with retries/logging).
- **Case Store:** Database (e.g. PostgreSQL) of cases, with each record tracking customer ID, merchant ID, case type, amount, creation time, status, last action, and metadata (attempts, last error code).
- **Decision Engine:** Core logic (could be a Python module with scheduled runner or an AI agent) that loads open cases and computes scores. Implements the Expected Net Recovery formula (see section 2).
- **Action Dispatcher:** Contains API clients and communication modules. Calls Razorpay’s APIs (payments retry, create Payment Links, etc.), sends emails/SMS via SMTP or a provider like SendGrid/Twilio, and updates case records.

{{ **Must-Build:** Robust webhook handling (signature check, idempotency keys, ordered processing) and a state machine in the Case Store. **Do Not Build:** A custom multi-agent framework or complex LLM chain—simple function or light orchestrator suffices. }}

2. Economic Decision Model

We frame each case as a **finite-horizon decision** with actions and an “exit” (STOP). Define:

- **Recoverable Value (V):** Money at risk for this case (invoice amount, cart total). Optionally weight by customer lifetime value.
- **Success Probability (p):** Likelihood an action succeeds (based on decline code patterns, historical data).
- **Cost (C):** Operational cost of the action (API call overhead, messaging cost).
- **Friction/Risk (F):** Negative impact of bothering the customer or merchant (estimated loss if annoyed).
- **Downside (D):** Penalty if action backfires (e.g. extra refund, disallowed card retries).

For a given action a , define its **Expected Net Recovery**:

$$\text{ENR}(a) = p(a) * V - C(a) - F(a) - D(a)$$

We choose the action *with the highest ENR* if it's > 0 ; otherwise STOP. As a result, STOP (no action) has $\text{ENR} = 0$ and is chosen when no profitable action remains. This is analogous to an *optimal stopping* rule [34†L26-L34] [29†L131-L139] .

Implementation details:

- **Inputs:** For each case, gather current state (amount, status), payment method, decline code, attempt count, customer payment history, time since failure.
- **Estimating p:** Use historical success rates by decline code, attempts, and possibly a simple ML classifier. (Stripe's and Recurly's docs suggest time-of-day and card info can refine p [38†L74-L80] [34†L26-L34] .) For MVP, use rule-of-thumb: e.g. soft declines $p \approx 0.7$ on first retry, p decays with attempts.
- **Costs/Friction:** Hard to quantify exactly. We can treat “1 email” as cost 1, “retry charge” cost minimal, “contacting merchant” cost higher. Customer friction could be a fixed penalty if we suspect over-contact (e.g. if last message was recent). These can be tuned.
- **Stopping rule:** After each action, recompute for the updated case. If all $\text{ENR}(a) \leq 0$, close case. (This ensures the agent “knows when to stop” before wasting resources.)

Pseudocode:

```
for case in open_cases:
    best_action = None
    best_score = 0
    for action in allowed_actions(case):
        p = estimate_success_prob(case, action)
        V = case.amount # or adjusted by CLV
        C = cost_of(action)
        F = friction_penalty(case, action)
```

```

D = risk_penalty(case, action)
score = p * V - (C + F + D)
if score > best_score:
    best_score = score
    best_action = action
if best_score <= 0:
    close_case(case)
else:
    execute_action(case, best_action)

```

{{ **Sources:** The expected-value approach is standard in decision theory. This agent is essentially maximizing expected utility (value minus cost) [29†L131-L139] [38†L74-L80] . The STOP decision is an *optimal stopping* rule. No existing product explicitly publishes this formula, so it's our novel contribution. }}

3. Razorpay Integration Checklist

Function	API/Endpoint	Doc/Notes	Used For
Create/ Update Payment Link	POST /v1/payment_links and PUT /v1/payment_links/{id}	Razorpay Payment Links API (official) [2†L101-L109] [3†L169-L178]	Action: generate new retry link or resend link
Retry a Charge	POST /v1/payments/{payment_id}/capture (for auth capture) or POST /v1/orders/{order_id}/payments/retry?	Razorpay Payments API (official) – Confirm exact endpoint.	Action: re- attempt a previously failed authorization
Fetch Payment Status	GET /v1/payments/{id}	Razorpay Payments API (official) – Check for full list	Check if previous retry succeeded or final status
Orders API	POST /v1/orders (PUT /v1/orders/{id})	Razorpay Orders API – create an order before capture (if needed)	If using Orders+Capture flow

Function	API/Endpoint	Doc/Notes	Used For
Invoices/ Receivables	POST /v1/invoices GET /v1/ invoices/{id} POST /v1/ invoices/{id}/notify POST /v1/invoices/{id}/complete	Razorpay Invoices API – to create/ manipulate invoices 【2†L108- L116】	For overdue receivables cases (generate/send invoice)
Subscriptions	POST /v1/subscriptions/{id}/ cancel PATCH /v1/ subscriptions/{id}	Razorpay Subscriptions API – manage failed recurring payments	Possibly to cancel a subscription if unrecoverable
Webhooks to Listen	e.g. payment.authorized, payment.failed, payment.captured, order.paid, invoice.paid, invoice.expired, subscription.charged_failed, payment_link.paid	Razorpay Webhooks Guide 【1†】 (and Webhooks Subscribe 【3†L208- L211】)	Ingestion of all events that create revenue risk cases
Webhook Signature	Use X-Razorpay-Signature header and validate with secret 【2†L143-L152】	Razorpay Webhook Signature docs – needed to verify authenticity.	Secure ingestion of events
Idempotency Keys	Include Idempotency-Key header on all POST requests	Razorpay Idempotency docs (to avoid duplicate retries)	Prevents double- charging during retries
Error Handling	Consult Razorpay API error codes (400/402) 【2†L145-L154】	Differentiate hard declines vs retryable errors.	Map errors to actions (e.g. stop on hard declines)
Rate Limits	~100 requests/minute (TBD) 【2†L145- L154】	Razorpay Rate Limits docs	Implement backoff/caching if approaching limit

Notes: Test-mode restrictions: Some Razorpay features (like sending email/SMS or live payouts) may not fully work in test-mode. We should simulate notifications and focus on API flow. *Live Verification:* We will need to try actual API calls with test keys to confirm behavior (especially for retries and webhooks).

{{Sources: We rely on Razorpay's official API references (dev docs) for exact endpoints. See Payment Links docs [2†L101-L109] , Webhooks docs [1†] , and Payments API docs (not directly fetchable, but summarized above).}}

4. Operational Engineering

Rate Limiting: Limit to ~100 API calls/min (Razorpay guideline). Use exponential backoff on 429. Batch actions (e.g. scheduling periodic polling) to avoid bursts. Cache static data (e.g. payment method details) to reduce calls.

Caching: Locally cache customer data or retry-policy info. However, keep “freshness” (e.g. don't cache a payment's failed status for too long). Use short-lived in-memory cache for non-sensitive info.

Idempotency & Duplicates: Always use `Idempotency-Key` for retry actions. Webhook events may arrive multiple times or out-of-order; track each event ID. Use de-duplication on case store to ignore repeats. Process events in order by timestamp when consistency matters (use queue or database ordering).

Consistency & Audit Logs: Each case update and decision must be transactional. Store every attempted action, outcome, timestamp, and agent rationale. Use ACID transactions in the database so that actions (API call) and logging succeed or fail together. Provide an audit log table for compliance.

Event-Driven Workflow: Use a durable message queue (e.g. RabbitMQ, AWS SQS) for webhooks → case updates. Ensure at-least-once delivery. After an action (e.g. retry), wait for resulting webhook (e.g. `payment.captured` or `payment.failed`) to update case, rather than polling.

Pagination: If fetching lists (e.g. list open invoices), handle Razorpay's pagination (`skip` , `count`) to avoid missing items. For our use-case, most operations are event-driven, but any batch job (e.g. periodic audit of cases) must page through API results or DB queries.

Failure Recovery: On crashes or partial failures, the queue and transactional DB ensure cases resume. E.g., if a retry API call succeeds but our service crashes before updating the case, the next webhook will still come and we mark success then.

{{Do Not Build: We will not implement a fully distributed microservices platform. A single-process worker or simple service with queue is enough.}}

5. Evaluation & Synthetic-Data Plan

Metric: *Net ₹ recovered per 100 cases.* For each batch of 100 simulated “at-risk cases”, run the agent and measure total money that would have been recovered minus any wasted costs. This is a clear scalar metric.

Data Schema: A synthetic case record has: `{customer_id, merchant_id, case_type, amount, created_at, decline_reason, attempts, status}`. We generate several case types:

- **Failed payments:** Vary amount (small to large), and decline codes (hard vs soft).
- **Abandoned carts:** e.g. uncompleted orders with user intent (represent as invoice).
- **Overdue invoices:** Unpaid invoice amounts with various ages. Each case also has a customer profile (CLV estimate, payment method list).

Failure Modes: For each case, define possible outcomes for each action probabilistically:

- If “retry”: succeed with probability p (lower if hard decline). If succeed, mark case closed; if fail, increment attempts and possibly change reason (simulate partial recovery, maybe next decline code).
- If “new link/email”: succeed if customer responds (probability q based on case data).
- If “do nothing”: remain failed (worst).
- Define friction: e.g. after each email, reduce q slightly to simulate fatigue.

Off-policy Evaluation: We cannot test live with real money, so we will simulate the environment. After the agent makes decisions, we apply the predetermined outcomes and compute final recovered amount. We'll compare our agent's strategy to baselines:

- *Fixed schedule:* e.g. try 3 times on fixed days, then email.
- *Naive max-attempt:* always use all attempts.
- *Stop-at-first:* alternate model. By comparing recovered and wasted costs, we quantify lift. Use multiple random seeds for fairness.

Counterfactual Aspect: To avoid “only see successful recoveries”, we fully simulate unchosen branches (e.g. if agent chooses STOP, we simulate what could have been recovered if it had tried – showing why STOP was justified).

Benchmark Procedure: 1. Generate $N=1000$ cases with random attributes. 2. Run our agent, record actions and outcomes. 3. Compute total initial at-risk ₹, recovered ₹, and waste (cost+friction). 4. Repeat with baseline policies on same data. 5. Report *net recovered* and *recovery rate*. 6. Also track % of cases correctly stopped (where further attempts would have net-negative effect), to demonstrate smart STOP logic.

{{**Sources:** Inspired by Recurly's findings that 90% of recoveries happen early [29†L36-L39] , we will emphasize early-window optimization. Our counterfactual setup is akin to “offline policy evaluation” in recommender systems research.}}

6. Competitive Analysis

Product / Solution	What it Solves	Limitations / Gap vs. Our Thesis	Source / Notes
Stripe Smart Retries	ML-driven timing for subscription/invoice retries. Uses time-of-day, device signals, rules.	Only covers Stripe invoices/subscriptions. No explicit “stop” decision (always retry up to X). No unified handling of non-invoice cases.	【38†L72-L80】 【38†L87-L94】
Recurly Intelligent Retries	ML-based optimal retry times for recurring credit card payments 【34†L26-L34】 . Increases success rates.	Limited to recurring credit cards. No built-in cross-case or stopping logic beyond, e.g., fixed attempt limit. Integrates with its own Dunning emails.	【34†L26-L34】 【34†L58-L66】
Recurly Dunning	Configurable email schedules and retry cycles for payment failures 【15†L12-L20】 【15†L139-L148】 .	Static and manual to configure; not data-driven. No unified EV model or automated “stop” decision (except final failure).	【15†L12-L20】 【15†L139-L148】
Chargebee Smart Retry	Dynamic retry intervals based on error type (hard/soft) 【17†L231-L239】 ; plus standard dunning emails.	No unified decision model; follows retries until fixed limit. Focused on cards/offline, lacks cross-case.	【17†L231-L239】
Baremetrics Recover	Automated email/SMS campaigns and paywalls layered on Stripe/others, triggered on failure 【21†L343-L352】 .	Focuses on communication (drip sequences); retries are left to Stripe. No decision optimization: fixed schedule of reminders. No integrated decision logic.	【21†L343-L352】 【21†L358-L366】
Razorpay Agent Studio (Subs)	“Subscription Recovery” agent: applies smarter retry logic and customer nudges 【24†L1-L4】 . Likely ML-based scheduling.	Possibly similar to Smart Retries. It’s limited to subscriptions; doesn’t merge with cart or invoices. No published EV model or broad STOP rule.	【24†L1-L4】
Razorpay Agent Studio (Others)	Also has abandoned cart agent (WhatsApp/email nudges) and others.	These are siloed. No single agent unifies payment failures + carts + receivables.	[23] marketing overview

Gap (Our Innovation): We found *no existing tool* that combines all revenue-risk types under one agent. All products use either static schedules or ML for timing, but **none** compute a net-benefit score to *decide whether to stop any further action*. Our thesis fills this gap with an explicit expected-value stopping criterion, a concept not claimed by competitors.

{{**Citations:** Stripe and Recurly docs [38†L72-L80] [34†L26-L34] , Recurly blog [29†L131-L139] , and vendor sites [21†L343-L352] [24†L1-L4] provide evidence of current capabilities and missing pieces.}}

7. Security & Compliance

- **Webhook Verification:** Validate Razorpay `X-Razorpay-Signature` on every webhook [2†L143-L152] . Reject unauthenticated requests.
- **Least Privilege:** Use a separate API key for the recovery agent with only necessary scopes (charges, links, invoices). Don't use a general account key.
- **Secret Management:** Store API keys and secrets in a secure vault (e.g. environment variables, AWS Secrets Manager). Never log raw secrets.
- **Idempotency & Race:** Protect against replay by tracking Razorpay `event_id` in DB. Use idempotency keys for POSTs to avoid duplicate charges.
- **PCI/PCI-DSS:** We will never handle raw card data. We only operate on tokens/payment method IDs provided by Razorpay. Our agent does not see card numbers or CVV. If storing customer contact info (email/phone), ensure it is encrypted at rest.
- **Audit Trail:** Log every decision with rationale. This is required by the AI agent standard. Ensure logs are tamper-evident (append-only log, or store hash chain).
- **Approval Gates:** By design, all actions are bounded (fixed retries, limited escalations). We won't let the agent make arbitrary transfers. The logic is transparent (no black-box), enabling easy human review if needed.
- **Data Privacy:** Follow GDPR/CCPA for customer communication. E.g. provide opt-out in emails, respect Do-Not-Contact flags. (This is more product design than tech, but keep in mind.)
- **Testing Environment:** In Razorpay test-mode, avoid contacting real customers. We can log the intended emails instead of sending to real addresses.

{{**Checklist:** Razorpay docs and PCI guidelines [2†L143-L152] should be reviewed before launch. Security isn't the core novelty, but careful engineering ensures we match Razorpay's standards.}}

8. Demo & Adversarial Test

Live Demo Script:

1. **Setup:** Use Razorpay test accounts. Pre-create a payment link for ₹1,500, deliberately then simulate failure:
2. **Trigger Case:** Simulate a failed payment event (e.g. simulate a `payment.failed` webhook with `status=failed` for an upcoming charge of ₹1,500). The agent creates a *case* (display on UI or log).
3. **Agent Decision:** The agent evaluates actions: it may choose "Retry Charge Now" since `p~0.8`, `V=1500`, `C~0`, showing expected net positive.
4. **Outcome:** Simulate the retry succeeding. The case closes. Output: "Recovered ₹1,500 after 1st retry (score X)".
5. **Second Case (Sequential Drift):** Now simulate a second case: a recurring subscription payment of ₹1,000 that keeps soft-failing. Show the agent retrying a few times, decrementing expected net each

time (because each retry has cost). Eventually, on the *third* failure, the model calculates that another retry has `ENR<=0` (perhaps due to reduced V or diminishing p) and chooses STOP. The log shows “STOP at attempt 3 (no more retries)”.

6. **Third Case (Mixed):** Show an abandoned cart of ₹2,000. The agent might first send a Payment Link (or send email/SMS) and schedule closure. Then simulate customer paying via the link (simulate webhook). Show case closed with ₹2,000 recovered.
7. **Outcome Display:** Show metrics: e.g. “Recovered ₹3,500 out of ₹4,500 in cases (78%) vs baseline 2 retries only (66%).” Also show audit trail in the log.

Adversarial Scenario:

- **Case:** Merchant’s UPI payment of ₹5,000 fails with a **hard decline code** (e.g. `authentication_required`). Our system should identify this as *non-retriable*.
- **Agent Behavior:** The agent should immediately STOP (since $p \approx 0$, and Stripe documentation confirms hard declines cannot be retried [38†L109-L118]).
- **Baseline Comparison:** A naive system (fixed retries) would waste attempts. We demonstrate the stop: log shows “Hard decline detected, STOP – no retry attempted.”
- **Evaluator Reaction:** This shows the agent respects payment-specific signals, not just “try 3 times then give up”. It highlights our differentiation: we integrated decline semantics into decision.

Impressiveness: A Razorpay evaluator will see that our demo not only recovers money, but also shows the agent *knowing when to quit* (a key differentiator). This should not look like just running Stripe Smart Retries; it’s a higher-level orchestration with explainable decisions.

9. Must-Read References

1. **Razorpay Payments & Webhooks Docs** – Official API docs for Payments, Payment Links, Webhooks (dev portal). *Needed for implementation.*
2. **Stripe Smart Retries Docs** [38†L72-L80] – Describes AI-driven retry logic and signals. *Use to benchmark and inspire timing model.*
3. **Recurly Intelligent Retries** [34†L26-L34] – Details ML retry engine for recurring cards. *Confirms ML approach and its limits.*
4. **Recurly Failed Payment Study** [29†L131-L139] – Data on recovery improvement with optimized retries. *Justifies our approach and metric.*
5. **Chargebee Dunning & Smart Retry** [17†L231-L239] – Shows Chargebee’s dynamic retry logic. *Confirms competitor capabilities.*
6. **Baremetrics Recover** [21†L343-L352] – Showcases email drip campaigns on top of retries. *Highlights communication gap.*
7. **Stanford Encyclopedia: Decision Theory** (SEU theory) – Background on expected utility (for modeling decisions). *Understand expected value maximization.*
8. **GDPR/PCI Standards** (e.g. official PCI-DSS site) – For compliance on handling payments and data. *Ensure secure implementation.*
9. **AI Agent Best Practices** (e.g. NIST AI RMF) – For auditability and governance. *Ensures we structure logs and explanations appropriately.*
10. **Optimal Stopping Theory** (textbook or survey) – The mathematical basis for STOP rule. *Conceptual rigor for our stopping logic.*

10. Competitor / Background Sources

1. **Stripe Smart Retries** – (docs.stripe.com) *What times-based ML retry does* [38†L72-L80] .
2. **Recurly Dunning & Retries** – (docs.recurly.com) *Their ML retry vs dunning workflows* [34†L26-L34] [15†L12-L20] .
3. **Chargebee Dunning** – (chargebee.com docs) *Smart retry intervals* [17†L231-L239] .
4. **Baremetrics Recover** – (baremetrics.com) *Email/SMS campaign integration* [21†L343-L352] .
5. **Razorpay Agent Studio** – (razorpay.com/agent-studio) *Subscription recovery agent description* [24†L1-L4] .

These are **key references** we will cite in our materials (docs, pitch, README) to show we surveyed the landscape and confirm our unique angle.
